Lab Exercise: Building Your First API-Connected LLM Agent

Objective

By the end of this lab, you will have built a Python-based LLM agent using LangChain and Google Gemini that can fetch real-time weather information via the wttr.in API.

Prerequisites

- Basic Python knowledge (functions, variables, installing packages)
- A Google Gemini API key
- A Python environment (Jupyter Notebook, VS Code, or Google Colab)

Part 1: Setting Up Your Environment

First, install the required libraries. We use langchain-google-genai to talk to Gemini, langchain classic for the agent runtime, and requests for API calls.

Run the following command in your terminal or a notebook cell:

```
pip install langchain langchain-classic langchain-google-genai langchain core requests python-dotenv
```

Next, create a .env file in your project folder and add your Gemini API key. Replace your-api-key-here with your actual secret key:

```
GEMINI_API_KEY=your-api-key-here
```

notes.arpanadhikari7.com.np

Save the file before running the script. You can get a key from Google AI Studio.

Alternatively, export the key in your shell:

```
export GEMINI_API_KEY=your-api-key-here
```

Part 2: Creating the API Tool

An agent needs "tools" (functions) it can run. Let's create a simple Python function that calls a free, no auth weather API (wttr.in) and wrap it so the LLM understands how to use it.

```
import requests
from langchain_core.tools import tool
```

```
@tool
def get_current_weather(location: str) -> str:
 """
 Fetches the current weather for a given city or location.  Use this tool whenever the
user asks about the weather.  """
 try:
 # We use ?format=3 for a clean, single-line text response (e.g., "Paris: ⛅ +12°C")
 url = f"https://wttr.in/{location}?format=3"
 response = requests.get(url)
 if response.status_code == 200:
 return response.text.strip()
 return "Could not retrieve weather data."
 except Exception as e:
 return f"Error fetching weather: {str(e)}"

# Combine our tool into a list
tools = [get_current_weather]
```

Why the docstring matters: Notice the text right under the function definition? The LLM reads that to decide when and how to use your tool. Be descriptive!

Part 3: Initializing the Agent

Now we hook up the Gemini model, define a custom prompt and initialize the agent executor.

```
import os
from pathlib import Path

from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder from
langchain_classic.agents import AgentExecutor,
create_tool_calling_agent
from langchain_google_genai import ChatGoogleGenerativeAI

def _load_gemini_api_key() -> str:
 """Load the Gemini API key from .env or shell environment."""
load_dotenv(Path(__file__).resolve().parent / ".env")
 api_key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")  if not api_key:
 raise ValueError(
 "Gemini API key not found. Add GEMINI_API_KEY to .env or export it."
 )
 return api_key

# 1. Initialize the LLM (Gemini 2.5 Flash for fast, cost-effective tasks) llm =
ChatGoogleGenerativeAI(
 model="gemini-2.5-flash",
```


Available on notes.arpanadhikari7.com.np

```
temperature=0,
api_key=_load_gemini_api_key(),
)

# 2. Define a custom prompt template for agent behavior
prompt = ChatPromptTemplate.from_messages(
[
(
"system",
"""You are a helpful assistant with access to tools.

When the user asks about current weather in a location, call
get_current_weather.
For general knowledge questions, answer directly without using tools. Think step by step, then
give a clear and concise final answer.""",  ),
MessagesPlaceholder("chat_history", optional=True),
("human", "{input}"),
MessagesPlaceholder("agent_scratchpad"),
]
)

# 3. Construct the tool-calling agent
agent = create_tool_calling_agent(llm, tools, prompt)

# 4. Create the executor (the runtime environment for the agent) agent_executor =
AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Part 4: Testing Your Agent

Time to see your creation in action! We will ask it two different questions: one that requires the API, and one that doesn't. Because verbose=True is enabled, you'll see the agent's "thought process" in the console.

Run the full script:

notes.arpanadhikari7.com.np

```
cd your_project_directory
python agent.py
```

Test 1: A general question (No API needed)

```
response = agent_executor.invoke({"input": "What is the capital of France?"})
print("\nFinal Answer:", response["output"])
```

What to expect: The agent should answer "Paris" immediately without trying to trigger the weather tool.

Test 2: A weather question (Triggers the API)

Available on notes.arpanadhikari7.com.np

```
response = agent_executor.invoke(
{"input": "Should I wear a coat in Tokyo today? Check the weather there."}
)
print("\nFinal Answer:", response["output"])
```

What to expect: You will see the agent realize it needs the weather, call `get_current_weather(location="Tokyo")`, receive the live temperature, and use that data to answer your coat question.

Lab Challenges to Try Next

Ready to take it a step further? Try modifying your code to handle these challenges:

- Change the prompt: Edit the `system` message in the `ChatPromptTemplate` so the agent answers like a grumpy pirate.
- Add a second tool: Create a tool that calculates the number of days until the upcoming weekend and add it to the `tools` list.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np